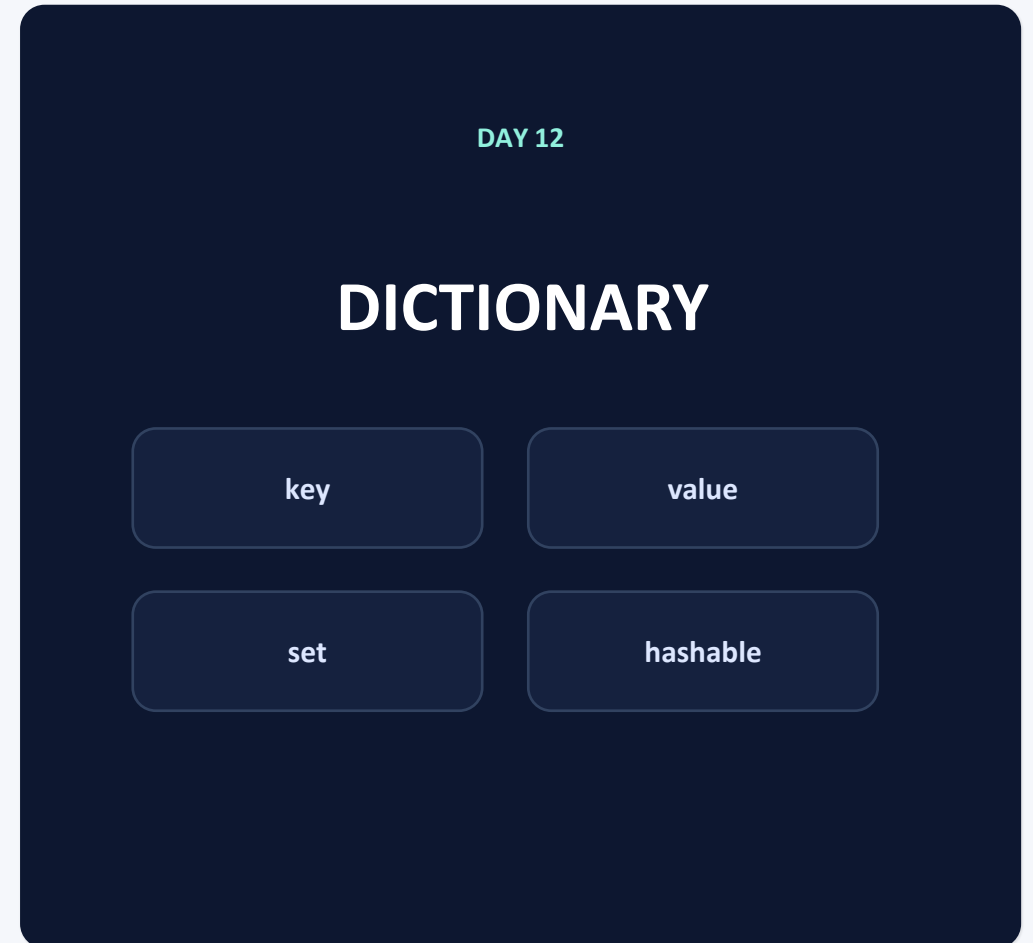


Словари и множества

Хранить данные по ключам в словаре и использовать множества для уникальности и быстрых операций над наборами.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

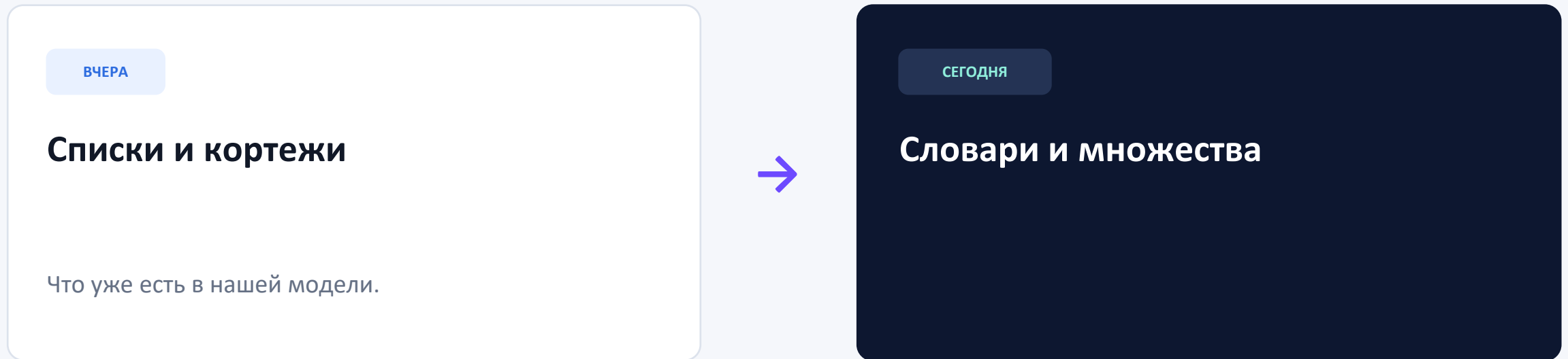
Конкретные действия вместо списка терминов.

- 01 Хранить данные по ключам в словаре и использовать множества для уникальности и быстрых операций над наборами.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Списки и кортежи. Сегодня мы добавляем следующий слой: словари и множества.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Когда важны не позиции 0,1,2, а смысловые имена полей — словарь делает код значительно понятнее. Множества помогают быстро убирать дубликаты и сравнивать наборы.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Словарь — карточка человека с подписанными полями; множество — коробка, где одинаковый предмет может лежать только один раз.

ПЕРЕВОД В IT

dictionary	коллекция пар ключ–значение
key	уникальный идентификатор значения в словаре
value	данные, сохранённые по ключу

Слой 1: dictionary • key • value

Сначала смысл, затем профессиональный термин.

**dictionary**

коллекция пар ключ–значение

**key**

уникальный идентификатор значения в словаре

**value**

данные, сохранённые по ключу

Слой 2: set • hashable • membership

Сначала смысл, затем профессиональный термин.

**set**

неупорядоченная коллекция уникальных элементов

**hashable**

пригодный для стабильного хеширования неизменяемый объект

**membership**

проверка наличия элемента оператором in

Слой 3: mapping • unique

Сначала смысл, затем профессиональный термин.

mapping

соответствие ключей их значениям

unique

свойство значения не повторяться в наборе

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: get • intersection

Сначала смысл, затем профессиональный термин.

get

безопасное получение значения словаря по ключу

intersection

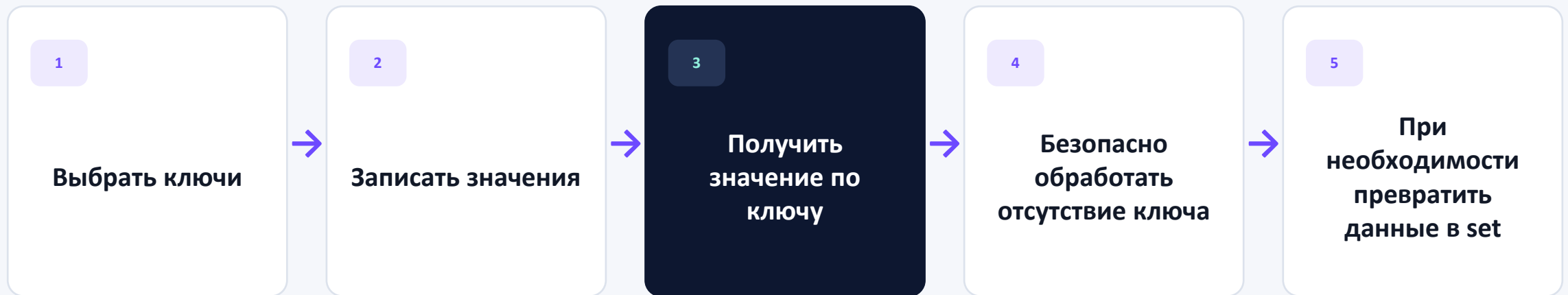
общие элементы двух множеств

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day12_step1

```
user = {"name": "Anna", "age": 25}  
print(user["name"])  
user["city"] = "Tallinn"  
print(user)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
user = {"name": "Anna", "age": 25}
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(user["name"])
```

Показываем результат, чтобы увидеть состояние программы.

```
user["city"] = "Tallinn"
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(user)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
user = {"name": "Anna", "age": 25}  
print(user["name"])  
user["city"] = "Tallinn"  
print(user)
```

РЕЗУЛЬТАТ

```
Anna  
{'name': 'Anna', 'age': 25, 'city':  
'Tallinn'}
```

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
skills = {"Python", "Git", "Python", "SQL"}  
print(len(skills))
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

3

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
user = {"name": "Anna"}  
print(user["email"])
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Ключа email нет, поэтому обращение через [] вызывает KeyError. Для необязательного поля удобнее get().

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
user = {"name": "Anna"}  
print(user.get("email", "не указан"))
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Найти общие навыки кандидата и вакансии с помощью множеств.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Создаём set навыков кандидата.**
- 2 Создаём set требований вакансии.
- 3 Берём пересечение оператором &.
- 4 Получаем только общие элементы.
- 5 Печатаем совпадения.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
candidate = {"Python", "Git", "Flask"}  
job = {"Python", "SQL", "Git"}  
common = candidate & job
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
candidate = {"Python", "Git", "Flask"}  
job = {"Python", "SQL", "Git"}  
common = candidate & job  
print(common)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

```
{'Python', 'Git'} # порядок элементов set может отличаться
```

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Собери словарь товара: name, price, quantity. Рассчитай сумму и добавь её как новое поле total.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Читай значения по ключам и записывай результат через `product["total"] = ...`

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
product = {"name": "Книга", "price": 20, "quantity": 3}  
product["total"] = product["price"] * product["quantity"]  
print(product["total"])
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

60

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Сравни два набора участников и найди тех, кто есть сразу в обеих группах.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
python_group = {"Anna", "Max", "Leo"}  
sql_group = {"Max", "Mia", "Anna"}  
print(python_group & sql_group)
```

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

```
{'Anna', 'Max'} # порядок может отличаться
```

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Ожидать фиксированный порядок от set.

02

Обращаться [] к необязательному ключу.

03

Использовать изменяемый объект как ключ словаря.

04

Путать значение и ключ.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «dictionary» и какую проблему это понятие помогает описать?

2

Чем «dictionary» отличается от «set»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **dictionary — коллекция пар ключ–значение.**

2 **dictionary: коллекция пар ключ–значение. set: неупорядоченная коллекция уникальных элементов.**

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **Anna**
`{'name': 'Anna', 'age': 25, 'city': 'Tallinn'}`

4 **Ключа `email` нет, поэтому обращение через `[]` вызывает `KeyError`. Для необязательного поля удобнее `get()`.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Хранить данные по ключам в словаре и использовать множества для уникальности и быстрых операций над наборами.
- ✓ Читать и объяснять пример темы «Словари и множества».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Строки и ввод пользователя

Завтра: Строки и ввод пользователя. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.